



TDL SDK C Interface

Copyright © 2025 CVITEK Co., Ltd. All rights reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of CVITEK Co., Ltd.

Contents

1	Model List	2
1.1	Object detection model list	2
1.2	Face detection model list	3
1.3	Face attribute and keypoint model list	3
1.4	Image classification model list	3
1.5	Sound classification model list	4
1.6	Keypoint model list	4
1.7	Lane detection model list	4
1.8	License plate recognition model list	4
1.9	Segmentation model list	5
1.10	Feature extraction model list	5
2	Structure Reference	6
2.1	TDLDataTypeE	6
2.2	TDLBox	7
2.3	TDLFeature	7
2.4	TDLPoints	8
2.5	TDLLandmarkInfo	8
2.6	TDLObjectInfo	9
2.7	TDLObject	9
2.8	TDLFaceInfo	10
2.9	TDLFace	11
2.10	TDLClassInfo	11
2.11	TDLClass	12
2.12	TDLKeypointInfo	12
2.13	TDLKeypoint	13
2.14	TDLSegmentation	13
2.15	TDLInstanceSegInfo	14
2.16	TDLInstanceSeg	14
2.17	TDLLanePoint	15
2.18	TDLLane	15
2.19	TDLDepthLogits	16
2.20	TDLTracker	16
2.21	TDLocr	17
3	API Reference	18
3.1	Handles	18
3.2	TDL_CreateHandle	18
3.3	TDL_CreateHandleEx	19
3.4	TDL_DestroyHandle	19
3.5	TDL_DestroyHandleEx	19

3.6	TDL_WrapVPSSFrame	20
3.7	TDL_ReadImage	20
3.8	TDL_ReadBin	21
3.9	TDL_DestroyImage	21
3.10	TDL_OpenModel	21
3.11	TDL_CloseModel	22
3.12	TDL_Detection	22
3.13	TDL_FaceDetection	23
3.14	TDL_FaceAttribute	23
3.15	TDL_FaceLandmark	24
3.16	TDL_Classification	25
3.17	TDL_ObjectClassification	26
3.18	TDL_KeypointDetection	26
3.19	TDL_InstanceSegmentation	27
3.20	TDL_SemanticSegmentation	28
3.21	TDL_FeatureExtraction	28
3.22	TDL_LaneDetection	29
3.23	TDL_DepthStereo	29
3.24	TDL_Tracking	30
3.25	TDL_CharacterRecognition	30
3.26	TDL_LoadModelConfig	31
3.27	TDL_SetModelDir	31
3.28	TDL_SetModelThreshold	32
3.29	TDL_IspClassification	32
3.30	TDL_Keypoint	33
3.31	TDL_DetectionKeypoint	34
3.32	TDL_IntrusionDetection	34
3.33	TDL_MotionDetection	35
3.34	TDL_APP_Init	36
3.35	TDL_APP_SetFrame	36
3.36	TDL_APP_Capture	37
3.37	TDL_APP_ConsumerCounting	37
3.38	TDL_WrapImage	38
3.39	TDL_LLMApiCall	39

Revision History

Date	Description	Owner
2025/03/28	Initial draft	Zhang Siyi
2025/03/31	Added API definitions	Liu Junfei
2025/04/02	Added model list and structure definitions	Zheng Xinye
2025/08/29	Updated model list and structure definitions	Zheng Xinye

1 Model List

1.1 Object detection model list

Model Name	Description
TDL_MODEL_MBV2_DET_PERSON	Human Detection Model(0:person)
TDL_MODEL_YOLOV8N_DET_HAND	Hand Detection Model(0:hand)
TDL_MODEL_YOLOV8N_DET_PET_PERSON	Pet and Person Detection Model (0:cat, 1:dog, 2:person)
TDL_MODEL_YOLOV8N_DET_PERSON_VEHICLE	Person and Vehicle Detection Model (0:car, 1:bus, 2:truck, 3:motorcyclist, 4:person, 5:bicycle, 6:motorcycle)
TDL_MODEL_YOLOV8N_DET_HAND_FACE_PERSON	Hand, Face and Person Detection Model (0:hand, 1:face, 2:person)
TDL_MODEL_YOLOV8N_DET_HEAD_PERSON	Head Detection Model (0:person, 1:head)
TDL_MODEL_YOLOV8N_DET_HEAD_HARDHAT	Head and Hardhat Detection Model (0:head, 1:hardhat)
TDL_MODEL_YOLOV8N_DET_FIRE_SMOKE	Fire and Smoke Detection Model (0:fire, 1:smoke)
TDL_MODEL_YOLOV8N_DET_FIRE	Fire Detection Model (0:fire)
TDL_MODEL_YOLOV8N_DET_HEAD_SHOULDER	Head and Shoulder Detection Model (0:head-shoulder)
TDL_MODEL_YOLOV8N_DET_LICENSE_PLATE	License Plate Detection Model (0:license plate)
TDL_MODEL_YOLOV8N_DET_TRAFFIC_LIGHT	Traffic Light Detection Model (0:red, 1:yellow, 2:green, 3:off, 4:waiting)
TDL_MODEL_YOLOV8N_DET_MONITOR_PERSON	Person Detection Model(0:person)
TDL_MODEL_YOLOV5_DET_COCO80	YOLOv5 COCO80 Detection Model
TDL_MODEL_YOLOV6_DET_COCO80	YOLOv6 COCO80 Detection Model
TDL_MODEL_YOLOV7_DET_COCO80	YOLOv7 COCO80 Detection Model
TDL_MODEL_YOLOV8_DET_COCO80	YOLOv8 COCO80 Detection Model
TDL_MODEL_YOLOV10_DET_COCO80	YOLOv10 COCO80 Detection Model
TDL_MODEL_PPYOLOE_DET_COCO80	PPYOLOE COCO80 Detection Model
TDL_MODEL_YOLOX_DET_COCO80	YOLOX COCO80 Detection Model

1.2 Face detection model list

Model Name	Description
TDL_MODEL_SCRFD_DET_FACE	Face Detection Model (0:face + keypoints)
TDL_MODEL_RETINA_DET_FACE	Face Detection Model
TDL_MODEL_RETINA_DET_FACE_IR	Infrared Face Detection Model

1.3 Face attribute and keypoint model list

Model Name	Description
TDL_MODEL_KEYPOINT_FACE_V2	Face Detection Model with 5 Keypoints and Blur Score
TDL_MODEL_CLS_ATTRIBUTE_GENDER _AGE_GLASS	Face Attribute Classification Model (age, gender, glasses)
TDL_MODEL_CLS_ATTRIBUTE_GENDER _AGE_GLASS_MASK	Face Attribute Classification Model (age, gender, glasses, mask)
TDL_MODEL_CLS_ATTRIBUTE_GENDER _AGE_GLASS_EMOTION	Face Attribute Classification Model (age, gender, glasses, emotion)

1.4 Image classification model list

Model Name	Description
TDL_MODEL_CLS_MASK	Mask Detection Model (0:wearing mask, 1:not wearing mask)
TDL_MODEL_CLS_RGBLIVENESS	Liveness Detection Model (0:real, 1:fake)
TDL_MODEL_CLS_ISP_SCENE	ISP Scene Classification Model
TDL_MODEL_CLS_HAND_GESTURE	Hand Gesture Classification Model (0:fist, 1:five fingers, 2:none, 3:two)
TDL_MODEL_CLS_KEYPOINT_HAND _GESTURE	Hand Gesture Keypoint Classification Model (0:fist, 1:five fingers, 2:four fingers, 3:none, 4:ok, 5:one, 6:three, 7:three2, 8:two)

1.5 Sound classification model list

Model Name	Description
TDL_MODEL_CLS_SOUND_BABAY_CRY	Baby Cry Sound Classification Model (0:background, 1:crying)
TDL_MODEL_CLS_SOUND_COMMAND_NIHAOSHIYUN	Command Sound Classification Model (0:background, 1:nihaoshiyun)
TDL_MODEL_CLS_SOUND_COMMAND_NIHAOSUANNENG	Command Sound Classification Model (0:background, 1:nihaosuanneng)
TDL_MODEL_CLS_SOUND_COMMAND_XIAOAIXIAOAI	Command Sound Classification Model (0:background, 1:xiaoaixiaoai)
TDL_MODEL_CLS_SOUND_COMMAND	Command Sound Classification Model

1.6 Keypoint model list

Model Name	Description
TDL_MODEL_KEYPOINT_LICENSE_PLATE	License Plate Keypoint Detection Model
TDL_MODEL_KEYPOINT_HAND	Hand Keypoint Detection Model
TDL_MODEL_KEYPOINT_YOLOV8POSE_PERSON17	Human 17 Keypoint Detection Model
TDL_MODEL_KEYPOINT_SIMCC_PERSON17	SIMCC 17 Keypoint Detection Model

1.7 Lane detection model list

Model Name	Description
TDL_MODEL_LSTR_DET_LANE	Lane Detection Model

1.8 License plate recognition model list

Model Name	Description
TDL_MODEL_RECOGNITION_LICENSE_PLATE	License Plate Recognition Model

1.9 Segmentation model list

Model Name	Descriptio
TDL_MODEL_YOLOV8_SEG_COCO80	YOLOv8 COCO80 Segmentation Model
TDL_MODEL_SEG_PERSON_FACE_VEHICLE	Person, Face and Vehicle Segmentation Model (0:background, 1:person, 2:face, 3:vehicle, 4:license plate)
TDL_MODEL_SEG_MOTION	Motion Segmentation Model (0:static, 2:transition, 3:motion)

1.10 Feature extraction model list

Model Name	Descriptio
TDL_MODEL_FEATURE_IMG	Image Feature Extraction Model
TDL_MODEL_IMG_FEATURE_CLIP	Image Clip Feature Extraction Model
TDL_MODEL_TEXT_FEATURE_CLIP	Text Clip Feature Extraction Model
TDL_MODEL_FEATURE_CVIFACE	cviface 256-dimensional Feature Extraction Model
TDL_MODEL_FEATURE_BMFACE_R34	ResNet34 512-dimensional Feature Extraction Model
TDL_MODEL_FEATURE_BMFACE_R50	ResNet50 512-dimensional Feature Extraction Model

2 Structure Reference

2.1 TDLDataTypeE

【Description】

Data Type Enumeration Class

【Definition】

```
typedef enum {
    TDL_TYPE_INT8 = 0, /**< Equals to int8_t. */
    TDL_TYPE_UINT8, /**< Equals to uint8_t. */
    TDL_TYPE_INT16, /**< Equals to int16_t. */
    TDL_TYPE_UINT16, /**< Equals to uint16_t. */
    TDL_TYPE_INT32, /**< Equals to int32_t. */
    TDL_TYPE_UINT32, /**< Equals to uint32_t. */
    TDL_TYPE_BF16, /**< Equals to bf17. */
    TDL_TYPE_FP16, /**< Equals to fp16. */
    TDL_TYPE_FP32, /**< Equals to fp32. */
    TDL_TYPE_UNKOWN /**< Equals to unkown. */
} TDLDataTypeE;
```

【Members】

Data Type Enumeration	Description
TDL_TYPE_INT8	Signed 8-bit integer
TDL_TYPE_UINT8	Unsigned 8-bit integer
TDL_TYPE_INT16	Signed 16-bit integer
TDL_TYPE_UINT16	Unsigned 16-bit integer
TDL_TYPE_INT32	Signed 32-bit integer
TDL_TYPE_UINT32	Unsigned 32-bit integer
TDL_TYPE_BF16	16-bit floating point (1 sign bit, 8 exponent bits, 7 mantissa bits)
TDL_TYPE_FP16	16-bit floating point (1 sign bit, 5 exponent bits, 10 mantissa bits)
FTDL_TYPE_FP32	32-bit floating point

2.2 TDLBox

【Description】

Box coordinate data

【Definition】

```
typedef struct {
    float x1;
    float y1;
    float x2;
    float y2;
} TDLBox;
```

【Members】

Data Type	Description
x1	x coordinate of top-left corner
y1	y coordinate of top-left corner
x2	x coordinate of bottom-right corner
y2	y coordinate of bottom-right corner

2.3 TDLFeature

【Description】

Feature value data

【Definition】

```
typedef struct {
    int8_t *ptr;
    uint32_t size;
    TDLDataTypeE type;
} TDLFeature;
```

【Members】

Data Type	Description
ptr	Feature value data
size	Data size
type	Data type

2.4 TDLPoints

【Description】

Coordinate queue data

【Definition】

```
typedef struct {
    float *x;
    float *y;
    uint32_t size;
    float score;
} TDLPoints;
```

【Members】

Data Type	Description
x	x coordinate queue data
y	y coordinate queue data
size	Size of coordinate queue
score	Score

2.5 TDLLandmarkInfo

【Description】

Feature point information

【Definition】

```
typedef struct {
    float x;
    float y;
    float score;
} TDLLandmarkInfo;
```

【Members】

Data Type	Description
x	x coordinate of feature point
y	y coordinate of feature point
score	Score

2.6 TDLObjectInfo

【Description】

Object detection information

【Definition】

```
typedef struct {
    TDLBox box;
    float score;
    int class_id;
    uint32_t landmark_size;
    TDLLandmarkInfo *landmark_properity;
    TDLObjectTypeE obj_type;
} TDLObjectInfo;
```

【Members】

Data Type	Description
score	Object detection score
class_id	Object detection class id
landmark_size	Size of object detection feature points
TDLLandmarkInfo	Object detection feature point information
obj_type	Object detection type

2.7 TDLObject

【Description】

Object detection data

【Definition】

```
typedef struct {
    uint32_t size;
    uint32_t width;
    uint32_t height;

    TDLObjectInfo *info;
} TDLObject;
```

【Members】

Data Type	Description
size	Number of detected objects
width	Width of detection image
height	Height of detection image
info	Object detection information

2.8 TDLFaceInfo

【Description】

Face information

【Definition】

```
typedef struct {
    char name[128];
    float score;
    uint64_t track_id;
    TDLBox box;
    TDLPoints landmarks;
    TDLFeature feature;

    float gender_score;
    float glass_score;
    float age;
    float liveness_score;
    float hardhat_score;
    float mask_score;

    float recog_score;
    float face_quality;
    float pose_score;
    float blurness;
} TDLFaceInfo;
```

【Members】

Data Type	Description
name	Face name
score	Face score
track_id	Face tracking id
box	Face box information
landmarks	Face feature points
feature	Face feature value
gender_score	Face gender score
glass_score	Whether wearing glasses
age	Face age
liveness_score	Face liveness score
hardhat_score	Face hardhat score
recog_score	Face recognition score
face_quality	Face quality score
pose_score	Face pose score
blurness	Face blur degree

2.9 TDLFace

【Description】

Face data

【Definition】

```
typedef struct {
    uint32_t size;
    uint32_t width;
    uint32_t height;
    TDLFaceInfo *info;
} TDLFace;
```

【Members】

Data Type	Description
size	Number of faces
width	Width of face image
height	Height of face image
info	Face information

2.10 TDLClassInfo

【Description】

Classification information

【Definition】

```
typedef struct {
    int32_t class_id;
    float score;
} TDLClassInfo;
```

【Members】

Data Type	Description
class_id	Classification class
score	Classification score

2.11 TDLClass

【Description】

Classification data

【Definition】

```
typedef struct {
    uint32_t size;
    TDLClassInfo *info;
} TDLClass;
```

【Members】

Data Type	Description
size	Number of classifications
info	Classification information

2.12 TDLKeypointInfo

【Description】

Keypoint information

【Definition】

```
typedef struct {
    float x;
    float y;
    float score;
} TDLKeypointInfo;
```

【Members】

Data Type	Description
x	x coordinate of keypoint
y	y coordinate of keypoint
score	Keypoint score

2.13 TDLKeypoint

【Description】

Keypoint data

【Definition】

```
typedef struct {
    uint32_t size;
    uint32_t width;
    uint32_t height;
    TDLKeypointInfo *info;
} TDLKeypoint;
```

【Members】

Data Type	Description
size	Number of keypoints
width	Image width
height	Image height
info	Keypoint information

2.14 TDLSegmentation

【Description】

Semantic segmentation data

【Definition】

```
typedef struct {
    uint32_t width;
    uint32_t height;
    uint32_t output_width;
    uint32_t output_height;
    uint8_t *class_id;
    uint8_t *class_conf;
} TDLSegmentation;
```

【Members】

Data Type	Description
width	Image width
height	Image height
output_width	Output image width
output_height	Output image height
class_id	Classification class
class_conf	Classification coordinate information

2.15 TDLInstanceSegInfo

【Description】

Instance segmentation information

【Definition】

```
typedef struct {
    uint8_t *mask;
    float *mask_point;
    uint32_t mask_point_size;
    TDLObjectInfo *obj_info;
} TDLInstanceSegInfo;
```

2.16 TDLInstanceSeg

【Description】

Instance segmentation data

【Definition】

```
typedef struct {
    uint32_t size;
    uint32_t width;
    uint32_t height;
    uint32_t mask_width;
    uint32_t mask_height;
    TDLInstanceSegInfo *info;
} TDLInstanceSeg;
```

【Members】

Data Type	Description
size	Number of instance segmentations
width	Image width
height	Image height
mask_width	Mask width
mask_height	Mask height
info	Instance segmentation information

2.17 TDLLanePoint

【Description】

Lane detection coordinate points

【Definition】

```
typedef struct {
    float x[2];
    float y[2];
    float score;
} TDLLanePoint;
```

【Members】

Data Type	Description
x	x coordinate queue
y	y coordinate queue
score	Lane detection score

2.18 TDLLane

【Description】

Lane detection data

【Definition】

```
typedef struct {
    uint32_t size;
    uint32_t width;
    uint32_t height;
    TDLLanePoint *lane;
    int lane_state;
} TDLLane;
```

【Members】

Data Type	Description
size	Number of lane detections
width	Image width
height	Image height
lane	Lane detection coordinate points
lane_state	Lane state

2.19 TDLDepthLogits

【Description】

Depth estimation data

【Definition】

```
typedef struct {
    int w;
    int h;
    int8_t *int_logits;
} TDLDepthLogits;
```

【Members】

Data Type	Description
w	Image width
h	Image height
int_logits	Depth estimation information

2.20 TDLTracker

【Description】

Tracking data

【Definition】

```
typedef struct {
    uint32_t size;
    uint64_t id;
    TDLBox bbox;
    int out_num;
} TDLTracker;
```

【Members】

Data Type	Description
size	Number of tracked targets
id	Tracking target ID
bbox	Tracking target bounding box
out_num	Number of times target is out of frame

2.21 TDLOcr

【Description】

Text recognition data

【Definition】

```
typedef struct {  
    uint32_t size;  
    char* text_info;  
} TDLOcr;
```

【Members】

Data Type	Description
size	Number of text recognitions
text_info	Text recognition information

3 API Reference

3.1 Handles

【Syntax】

```
typedef void *TDLHandle;  
typedef void *TDLImage;
```

【Description】

TDL SDK handles, TDLHandle is the core operation handle, TDLImage is the image data abstraction handle.

3.2 TDL_CreateHandle

【Syntax】

```
TDLHandle TDL_CreateHandle(const int32_t tpu_device_id);
```

【Description】

Create a TDLHandle object.

【Parameters】

	Data Type	Parameter Name	Description
Input	const int32_t	tpu_device_id	specified TPU device ID

3.3 TDL_CreateHandleEx

【Syntax】

```
TDLHandleEx TDL_CreateHandleEx(const int32_t tpu_device_id);
```

【Description】

Create a TDLHandleEx object.

【Parameters】

	Data Type	Parameter Name	Description
Input	const int32_t	tpu_device_id	specified TPU device ID

3.4 TDL_DestroyHandle

【Syntax】

```
int32_t TDL_DestroyHandle(TDLHandle handle);
```

【Description】

Destroy a TDLHandle object.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object to be destroyed

3.5 TDL_DestroyHandleEx

【Syntax】

```
int32_t TDL_DestroyHandle(TDLHandle handle);
```

【Description】

Destroy a TDLHandleEx object.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandleEx object to be destroyed

3.6 TDL_WrapVPSSFrame

【Syntax】

```
TDLImage TDL_WrapVPSSFrame(void *vpss_frame, bool own_memory);
```

【Description】

Wrap a VPSS frame as a TDLImageHandle object.

【Parameters】

	Data Type	Parameter Name	Description
Input	void*	vpss_frame	VPSS frame to be wrapped
Input	bool	own_memory	Whether to own the memory

3.7 TDL_ReadImage

【Syntax】

```
TDLImage TDL_ReadImage(const char *path);
```

【Description】

Read an image as a TDLImageHandle object.

【Parameters】

	Data Type	Parameter Name	Description
Input	const char*	path	Image path

3.8 TDL_ReadBin

【Syntax】

```
TDLImage TDL_ReadBin(const char *path, int count, TDLDataTypeE data_type);
```

【Description】

Read file content as a TDLImageHandle object.

【Parameters】

	Data Type	Parameter Name	Description
Input	const char*	path	Binary file path
Input	int	count	Data count in file
Input	TDLDataTypeE	data_type	Input data type

3.9 TDL_DestroyImage

【Syntax】

```
int32_t TDL_DestroyImage(TDLImage image_handle);
```

【Description】

Destroy a TDLImageHandle object.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLImage	image_handle	TDLImageHandle object to be destroyed

3.10 TDL_OpenModel

【Syntax】

```
int32_t TDL_OpenModel(TDLHandle handle,
                      const TDLModel model_id,
                      const char *model_path);
```


【Description】

Load a specified type of model into the TDLHandle object.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const TDLModel	model_id	Model type enumeration
Input	const char*	model_path	Model path

3.11 TDL_CloseModel

【Syntax】

```
int32_t TDL_CloseModel(TDLHandle handle,
                       const TDLModel model_id);
```

【Description】

Unload the specified type of model and release related resources.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const TDLModel	model_id	Model type enumeration

3.12 TDL_Detection

【Syntax】

```
int32_t TDL_Detection(TDLHandle handle,
                     const TDLModel model_id,
                     TDLImage image_handle,
                     TDLObject *object_meta);
```

【Description】

Execute inference detection with the specified model and return detection result metadata.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const TDLModel	model_id	Model type enumeration
Input	TDLImage	image_handle	TDLImageHandle object
Output	TDLObject*	object_meta	Output detection result metadata

3.13 TDL_FaceDetection

【Syntax】

```
int32_t TDL_FaceDetection(TDLHandle handle,
                          const TDLModel model_id,
                          TDLImage image_handle,
                          TDLFace *face_meta);
```

【Description】

Execute face detection and return face detection result metadata.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const TDLModel	model_id	Model type enumeration
Input	TDLImage	image_handle	TDLImageHandle object
Output	TDLFace*	face_meta	Output face detection result metadata

3.14 TDL_FaceAttribute

【Syntax】

```
int32_t TDL_FaceAttribute(TDLHandle handle,
                          const TDLModel model_id,
                          TDLImage image_handle,
                          TDLFace *face_meta);
```

【Description】

Execute face attribute analysis, requires face detection results for feature analysis.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const TDLModel	model_id	Model type enumeration
Input	TDLImage	image_handle	TDLImageHandle object
Input/Output	TDLFace*	face_meta	Input face detection results, output additional attribute information

3.15 TDL_FaceLandmark

【Syntax】

```
int32_t TDL_FaceLandmark(TDLHandle handle,
                        const TDLModel model_id,
                        TDLImage image_handle,
                        TDLImage *crop_image_handle,
                        TDLFace *face_meta);
```

【Description】

Execute face keypoint detection, supplementing keypoint coordinates to existing face detection results.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const TDLModel	model_id	Model type enumeration
Input	TDLImage	image_handle	TDLImageHandle object
Input	TDLImage	crop_image_handle	TDLImageHandle object, the cropped image. It will not take effect if it is NULL.
Input/Output	TDLFace*	face_meta	Input face detection results, output additional keypoint coordinates

3.16 TDL_Classification

【Syntax】

```
int32_t TDL_Classification(TDLHandle handle,
                          const TDLModel model_id,
                          TDLImage image_handle,
                          TDLClassInfo *class_info);
```

【Description】

Execute general classification recognition.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const TDLModel	model_id	Model type enumeration
Input	TDLImage	image_handle	TDLImageHandle object
Output	TDLClassInfo*	class_info	Output classification results

3.17 TDL_ObjectClassification

【Syntax】

```
int32_t TDL_ObjectClassification(TDLHandle handle,
                                const TDLModel model_id,
                                TDLImage image_handle,
                                TDLObject *object_meta,
                                TDLClass *class_info);
```

【Description】

Perform fine-grained classification on detected objects.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const TDLModel	model_id	Model type enumeration
Input	TDLImage	image_handle	TDLImageHandle object
Input	TDLObject*	object_meta	Detected object information
Output	TDLClass*	class_info	Output object classification results

3.18 TDL_KeypointDetection

【Syntax】

```
int32_t TDL_KeypointDetection(TDLHandle handle,
                              const TDLModel model_id,
                              TDLImage image_handle,
                              TDLKeypoint *keypoint_meta);
```

【Description】

Perform human/object keypoint detection.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const TDLModel	model_id	Model type enumeration
Input	TDLImage	image_handle	TDLImageHandle object
Output	TDLKeypoint*	keypoint_meta	Output keypoint coordinates and confidence

3.19 TDL_InstanceSegmentation

【Syntax】

```
int32_t TDL_InstanceSegmentation(TDLHandle handle,
                                const TDLModel model_id,
                                TDLImage image_handle,
                                TDLInstanceSeg *inst_seg_meta);
```

【Description】

Perform instance segmentation (Instance Segmentation), detecting the pixel-level contours of each separate object in the image.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const TDLModel	model_id	Model type enumeration
Input	TDLImage	image_handle	TDLImageHandle object
Output	TDLInstanceSeg*	inst_seg_meta	Output instance segmentation results (including mask and bbox)

3.20 TDL_SemanticSegmentation

【Syntax】

```
int32_t TDL_SemanticSegmentation(TDLHandle handle,
                                const TDLModel model_id,
                                TDLImage image_handle,
                                TDLSegmentation *seg_meta);
```

【Description】

Perform semantic segmentation (Semantic Segmentation), classifying each pixel in the image.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const TDLModel	model_id	Model type enumeration
Input	TDLImage	image_handle	TDLImageHandle object
Output	TDLSegmentation*	seg_meta	Output segmentation results (label map)

3.21 TDL_FeatureExtraction

【Syntax】

```
int32_t TDL_FeatureExtraction(TDLHandle handle,
                              const TDLModel model_id,
                              TDLImage image_handle,
                              TDLFeature *feature_meta);
```

【Description】

Extract deep feature vectors from the image.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const TDLModel	model_id	Model type enumeration
Input	TDLImage	image_handle	TDLImageHandle object
Output	TDLFeature*	feature_meta	Output feature vector

3.22 TDL_LaneDetection

【Syntax】

```
int32_t TDL_LaneDetection(TDLHandle handle,
                          const TDLModel model_id,
                          TDLImage image_handle,
                          TDLLane *lane_meta);
```

【Description】

Detect lane lines and their attributes.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const TDLModel	model_id	Model type enumeration
Input	TDLImage	image_handle	TDLImageHandle object
Output	TDLLane*	lane_meta	Output lane line coordinates and attributes

3.23 TDL_DepthStereo

【Syntax】

```
int32_t TDL_DepthStereo(TDLHandle handle,
                        const TDLModel model_id,
                        TDLImage image_handle,
                        TDLDepthLogits *depth_logist);
```

【Description】

Depth estimation based on stereo vision, outputting depth confidence map.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const TDLModel	model_id	Model type enumeration
Input	TDLImage	image_handle	TDLImageHandle object
Output	TDLDepthLogits*	depth_logist	Output depth confidence data

3.24 TDL_Tracking

【Syntax】

```
int32_t TDL_Tracking(TDLHandle handle,
                    const TDLModel model_id,
                    TDLImage image_handle,
                    TDLObject *object_meta,
                    TDLTracker *tracker_meta);
```

【Description】

Multi-object tracking, associating detected objects across frames.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const TDLModel	model_id	Model type enumeration
Input	TDLImage	image_handle	TDLImageHandle object
In-put/Output	TDLObject*	object_meta	Input detection results, output tracking IDs
Output	TDLTracker*	tracker_meta	Output tracker status information

3.25 TDL_CharacterRecognition

【Syntax】

```
int32_t TDL_CharacterRecognition(TDLHandle handle,
                                const TDLModel model_id,
                                TDLImage image_handle,
                                TDLOcr *char_meta);
```

【Description】

Character recognition, supporting text detection and recognition.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const TDLModel	model_id	Model type enumeration
Input	TDLImage	image_handle	TDLImageHandle object
Output	TDLOcr*	char_meta	Output recognition results (text content and position)

3.26 TDL_LoadModelConfig

【Syntax】

```
int32_t TDL_LoadModelConfig(TDLHandle handle,
                           const char *model_config_json_path);
```

【Description】

Load model configuration information, after loading you can open models using only model IDs.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const char*	model_config_json_path	Model configuration file path, if NULL, defaults to configs/model/model_config.json

3.27 TDL_SetModelDir

【Syntax】

```
int32_t TDL_SetModelDir(TDLHandle handle,
                       const char *model_dir);
```

【Description】

Set the model directory path.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const char*	model_dir	Path to tdl_models repository (sub-folders for different platforms)

3.28 TDL_SetModelThreshold

【Syntax】

```
int32_t TDL_SetModelThreshold(TDLHandle handle,
                             const TDLModel model_id,
                             float threshold);
```

【Description】

Set the model threshold value.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const TDLModel	model_id	Model type enumeration
Input	float	threshold	Model threshold value

3.29 TDL_IspClassification

【Syntax】

```
int32_t TDL_IspClassification(TDLHandle handle,
                              const TDLModel model_id,
                              TDLImage image_handle,
                              TDLIspMeta *isp_meta,
                              TDLClass *class_info);
```

【Description】

Execute ISP image classification task.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const TDLModel	model_id	Model type enumeration
Input	TDLImage	image_handle	TDLImageHandle object
Input	TDLispMeta*	isp_meta	Input ISP related data
Output	TDLClass*	class_info	Output classification results

3.30 TDL_Keypoint

【Syntax】

```
int32_t TDL_Keypoint(TDLHandle handle,
                    const TDLModel model_id,
                    TDLImage image_handle,
                    TDLKeypoint *keypoint_meta);
```

【Description】

Execute keypoint detection task.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const TDLModel	model_id	Model type enumeration
Input	TDLImage	image_handle	TDLImageHandle object
Output	TDLKeypoint*	keypoint_meta	Output detected keypoint coordinates and confidence

3.31 TDL_DetectionKeypoint

【Syntax】

```
int32_t TDL_DetectionKeypoint(TDLHandle handle,
                              const TDLModel model_id,
                              TDLImage image_handle,
                              TDLObject *object_meta);
```

【Description】

Execute keypoint detection task based on object coordinates (performs keypoint detection after cropping based on target coordinates).

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const TDLModel	model_id	Model type enumeration
Input	TDLImage	image_handle	TDLImageHandle object
Output	TDLObject*	object_meta	Output detected keypoint coordinates and confidence

3.32 TDL_IntrusionDetection

【Syntax】

```
int32_t TDL_IntrusionDetection(TDLHandle handle,
                                TDLPoints *regions,
                                TDLBox *box,
                                bool *is_intrusion);
```

【Description】

Execute intrusion detection.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	TDLPoints*	regions	Background region point set array
Input	TDLBox*	box	Detection region bbox
Output	bool*	is_intrusion	Output intrusion detection result

3.33 TDL_MotionDetection

【Syntax】

```
int32_t TDL_MotionDetection(TDLHandle handle,
    TDLImage background,
    TDLImage detect_image,
    TDLObject *roi,
    uint8_t threshold,
    double min_area,
    TDLObject *obj_meta);
```

【Description】

Execute motion detection task.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	TDLImage	back-ground	Background image
Input	TDLImage	detect_image	Detection image
Input	TDLObject*	roi	Detection region
Input	uint8_t	threshold	Threshold value
Input	double	min_area	Minimum area
Output	TDLObject*	obj_meta	Output detection results

3.34 TDL_APP_Init

【Syntax】

```
int32_t TDL_APP_Init(TDLHandle handle,
                    const char *task,
                    const char *config_file,
                    char ***channel_names,
                    uint8_t *channel_size);
```

【Description】

Initialize APP task.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const char*	task	APP task name
Input	const char*	config_file	APP json configuration file path
Output	char***	channel_names	Name information for each video stream
Output	uint8_t*	channel_size	Number of video streams

3.35 TDL_APP_SetFrame

【Syntax】

```
int32_t TDL_APP_SetFrame(TDLHandle handle,
                        const char *channel_name,
                        TDLImage image_handle,
                        uint64_t frame_id,
                        int buffer_size);
```

【Description】

Send frame to APP.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const char*	channel_name	Current channel name
Input	TDLImage	image_handle	TDLImageHandle object
Input	uint64_t	frame_id	Frame ID of current TDLImageHandle object
Input	int	buffer_size	Number of frames cached by inference thread

3.36 TDL_APP_Capture

【Syntax】

```
int32_t TDL_APP_Capture(TDLHandle handle,
                        const char *channel_name,
                        TDLCaptureInfo *capture_info);
```

【Description】

Execute face capture task.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const char*	channel_name	Current channel name
Output	TDLCaptureInfo*	capture_info	Capture results

3.37 TDL_APP_ConsumerCounting

【Syntax】

```
int32_t TDL_APP_ConsumerCounting(TDLHandle handle,
                                  const char *channel_name,
                                  TDLObject *object_meta,
```

(continues on next page)

(continued from previous page)

```
uint32_t *enter_num,
uint32_t *miss_num);
```

【Description】

Execute consumer counting task.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	TDLHandle object
Input	const char*	channel_name	Current channel name
Input	TDLObject*	object_meta	Detection results
Output	uint32_t*	enter_num	Number of people entering
Output	uint32_t*	miss_num	Number of people leaving

3.38 TDL_WrapImage

【Syntax】

```
int32_t TDL_WrapImage(TDLImage image,
                      VIDEO_FRAME_INFO_S *frame);
```

【Description】

Wrap TDLImage as VIDEO_FRAME_INFO_S.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLImage	image	TDLImageHandle object
Output	VIDEO_FRAME_INFO_S*	frame	Output parameter, stores the wrapped frame information

3.39 TDL_LLMApiCall

【Syntax】

```
int32_t TDL_LLMApiCall(TDLHandle handle, const char *client_type,
                      const char *method_name, const char *params_json,
                      char *result_buf, size_t buf_size)
```

【Description】

Invoke a specific LLM client method.

【Parameters】

	Data Type	Parameter Name	Description
Input	TDLHandle	handle	Context handle returned by TDL_CreateHandle
Input	const char*	client_type	Specifies the type of LLM client to invoke
Input	const char*	method_name	Name of the API method to call
Input	const char*	params_json	JSON-formatted request body string
Input	size_t	buf_size	Available size of result_buf in bytes
Output	char*	result_buf	Buffer for storing the returned JSON result or error message